

Algorithms and Data Structures

27.01.2025

Exercise 1 (4.0 points)

Concerning dynamic memory allocation in the C language, describe how you can define two-dimensional matrices of integer types. First, clarify how to store a 2D matrix into a 1D array and logically move from one data structure to the other. Then, describe how to perform dynamic memory allocation for 2D matrices.

Solution

Please see the teacher's overhead for a complete analysis of the topic.

Exercise 2 (1.5 points)

Given the following array of integer values, perform the first step of quicksort to sort the array in ascending order; then, from the initial array, generate the right and the left partitions.

11 12 13 5 8 0 2 3 1 7

Report three integer values: The pivot selected on the original array, the pivot you would select on the left partition generated from the original array, and the pivot you would select on the right partition generated (again) from the original array. No other symbols must be included in the response. The following is an example of the response format:

13 1 10

Solution

Quick sort

Solution format 1:

```
Initial array: 11 12 13 5 8 0 2 3 1 7
Re l= 0;r= 4: 1 3 2 5 0
Re l= 0;r=-1:
Re l= 0;r=-1: 3 2 5 1
Re l= 1;r= 0:
Re l= 1;r= 0: 2 5 3
Re l= 2;r= 2: 2
Re l= 2;r= 2: 5
Re l= 0;r= 4: 13 12 11 8
Re l= 6;r= 5:
Re l= 6;r= 5: 12 11 13
Re l= 7;r= 8: 12 11
Re l= 7;r= 6: 12
Re l= 7;r= 6:
Re l= 7;r= 8:
```

Solution format 2:

```
Input array: [0]11 [1]12 [2]13 [3]5 [4]8 [5]0 [6]2 [7]3 [8]1 [9]7
Pivot: [9]7
Swaps: [0<->8]11<->1 - [1<->7]12<->3 - [2<->6]13<->2 - [4<->5]8<->0 - [5<->9]8<->7
Output array: [0]1 [1]3 [2]2 [3]5 [4]0 - [5]7 - [6]13 [7]12 [8]11 [9]8
Input array: [0]1 [1]3 [2]2 [3]5 [4]0 - [6]13 [7]12 [8]11 [9]8
Pivot: [4]0 - [9]8
Swaps: [0<->4]1<->0 - [6<->9]13<->8
Output array: - [0]0 - [1]3 [2]2 [3]5 [4]1 - - [6]8 - [7]12 [8]11 [9]13
Input array: [1]3 [2]2 [3]5 [4]1 - [7]12 [8]11 [9]13
Pivot: [4]1 - [9]13
Swaps: [1<->4]3<->1 - [9<->9]13<->13
Output array: - [1]1 - [2]2 [3]5 [4]3 - [7]12 [8]11 - [9]13 -
Input array: [2]2 [3]5 [4]3 - [7]12 [8]11
Pivot: [4]3 - [8]11
Swaps: [3<->4]5<->3 - [7<->8]12<->11
Output array: [2]2 - [3]3 - [4]5 - - [7]11 - [8]12
```

Response

7 0 8

Exercise 3 (1.5 points)

Consider a binary tree whose visits return the following sequences.

```
Pre-order: A C D I G K B E H F J
In-order: I D C K G A E H B F J
Post-order: I D K G C H E J F B A
```

Report the sequence of keys stored on the tree leaves, moving on the tree from left to right. Please report the list of keys on the same line, separated by a single space. No other symbols must be included in the response. The following is an example of the response format: A X C Y etc.

Solution

```
### Tree guess
Node Key  Left  Right
1      A    2    3
2      C    4    5
3      B    6    7
4      D    8  NULL
5      G    9  NULL
6      E  NULL 10
7      F  NULL 11
8      I  NULL NULL
9      K  NULL NULL
10     H  NULL NULL
11     J  NULL NULL
```

Response

I K H J

Exercise 4 (1.5 points)

Insert the following sequence of keys into an initially empty hash table. The hash table has a size equal to $M=23$. Insertions occur character by character using open addressing with quadratic probing (with $c_1 = 1$ and $c_2 = 1$). Each character is identified by its index in the English alphabet (i.e., $A=1, \dots, Z=26$). Equal letters are identified by a different subscript (i.e., A and A become A1 and A2).

G I G L I N G

Indicate in which elements the last three letters of the sequence are placed, i.e., I, N, and G, in this order. Please report your response as a sequence of integer values separated by one single space. No other symbols must be included in the response. The following is an example of the response format: 3 4 11

Solution

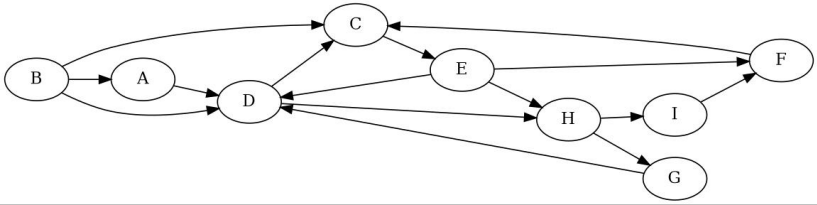
```
### hash-table
Character keys
Hash table size = 23
Number of keys  = 7
Quadratic probing: c1=1.000000, c2=1.000000.
Key:  G I G L I N G
Key=G k=007 Final=07
Key=I k=009 Final=09
Key=G k=007 Coll.= 7 Coll.= 9 Final=13
Key=L k=012 Final=12
Key=I k=009 Coll.= 9 Final=11
Key=N k=014 Final=14
Key=G k=007 Coll.= 7 Coll.= 9 Coll.=13 Final=19
00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22
-  -  -  -  -  -  -  G  -  I  -  I  L  G  N  -  -  -  -  G  -  -  -
```

Response

11 14 19

Exercise 5 (1.5 points)

Visit the following graph depth-first, starting at node A. Label nodes with discovery and end-processing times. Start with the discovery time set to 1 on A. When necessary, consider nodes and edges in alphabetic order.



Display the end-processing time of all vertices. Please indicate the end-processing time of all vertices sorted in alphabetic order (i.e., display the end-processing time for A B C D etc.). Report a sequence of integer values separated

by one single space. No other symbols must be included in the response. The following is an example of the response:
15 13 2 16 8 etc.

Solution

```

Directed matrix ... stated as such.
UN-weighted matrix ... stated as such.
Vertex list: [ 0 ] A [ 1 ] B [ 2 ] C [ 3 ] D [ 4 ] E [ 5 ] F [ 6 ] G [ 7 ] H [ 8 ] I
Adjacency Matrix:
0 0 0 1 0 0 0 0 0
1 0 1 1 0 0 0 0 0
0 0 0 0 1 0 0 0 0
0 0 1 0 0 0 0 1 0
0 0 0 1 0 1 0 1 0
0 0 1 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0
0 0 0 0 0 1 0 1 1
0 0 0 0 0 1 0 0 0

### Graph visit
BFS:
Node=[ 0 ] A Discovery_time= 0 Predecessor=[-1]
Node=[ 1 ] B Discovery_time= - Predecessor=[-1]
Node=[ 2 ] C Discovery_time= 2 Predecessor=[ 3]
Node=[ 3 ] D Discovery_time= 1 Predecessor=[ 0]
Node=[ 4 ] E Discovery_time= 3 Predecessor=[ 2]
Node=[ 5 ] F Discovery_time= 4 Predecessor=[ 4]
Node=[ 6 ] G Discovery_time= 3 Predecessor=[ 7]
Node=[ 7 ] H Discovery_time= 2 Predecessor=[ 3]
Node=[ 8 ] I Discovery_time= 3 Predecessor=[ 7]

DFS:
List of edges:
[ 0 ] A -> [ 3 ] D : (T) Tree
[ 3 ] D -> [ 2 ] C : (T) Tree
[ 2 ] C -> [ 4 ] E : (T) Tree
[ 4 ] E -> [ 3 ] D : (B) Back
[ 4 ] E -> [ 5 ] F : (T) Tree
[ 5 ] F -> [ 2 ] C : (B) Back
[ 4 ] E -> [ 7 ] H : (T) Tree
[ 7 ] H -> [ 6 ] G : (T) Tree
[ 6 ] G -> [ 3 ] D : (B) Back
[ 7 ] H -> [ 8 ] I : (T) Tree
[ 8 ] I -> [ 5 ] F : (C) Cross
[ 3 ] D -> [ 7 ] H : (F) Forward
[ 1 ] B -> [ 0 ] A : (C) Cross
[ 1 ] B -> [ 2 ] C : (C) Cross
[ 1 ] B -> [ 3 ] D : (C) Cross
List of vertices:
[ 0 ] A: dist_time= 1 endp_time=16 pred=[-1] -
[ 1 ] B: dist_time=17 endp_time=18 pred=[-1] -
[ 2 ] C: dist_time= 3 endp_time=14 pred=[ 3] D
[ 3 ] D: dist_time= 2 endp_time=15 pred=[ 0] A
[ 4 ] E: dist_time= 4 endp_time=13 pred=[ 2] C
[ 5 ] F: dist_time= 5 endp_time= 6 pred=[ 4] E
[ 6 ] G: dist_time= 8 endp_time= 9 pred=[ 7] H
[ 7 ] H: dist_time= 7 endp_time=12 pred=[ 4] E
[ 8 ] I: dist_time=10 endp_time=11 pred=[ 7] H
Reachable nodes from vertex [ 0 ] A:
[ 0 ] A - [ 1 ] B - [ 2 ] C - [ 3 ] D - [ 4 ] E - [ 5 ] F - [ 6 ] G - [ 7 ] H - [ 8 ] I
UN-reachable nodes from vertex [ 0 ] A:
none

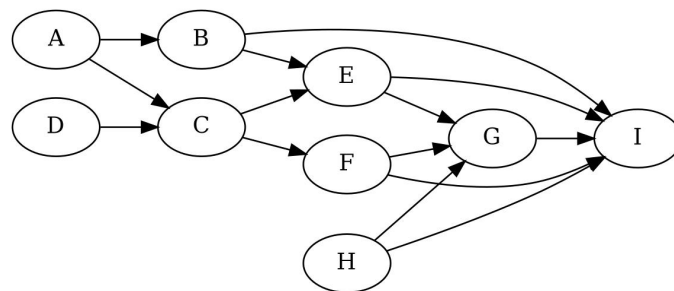
```

Response

16 18 14 15 13 6 9 12 11

Exercise 6 (1.5 points)

Given the following unweighted DAG, find the topological order of its vertices. Start the DFS visit from vertex A. If necessary, consider nodes in alphabetical order.



Report the direct topological order of the vertices. Report vertices on the same line separated by a single space. No

other symbols must be included in the response. The following is an example of the response format: F A C H B etc.

Solution

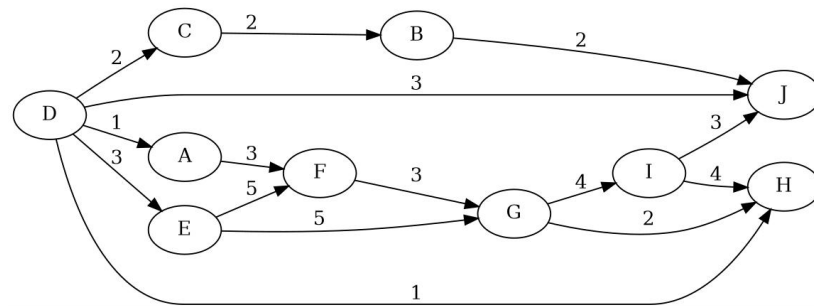
```
Directed matrix ... stated as such.
UN-weighted matrix ...stated as such.
Vertex list: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 5] F [ 6] G [ 7] H [ 8] I
Adjacency Matrix:
0 1 1 0 0 0 0 0 0
0 0 0 0 1 0 0 0 1
0 0 0 0 1 1 0 0 0
0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 1 0 1
0 0 0 0 0 0 1 0 1
0 0 0 0 0 0 1 0 1
0 0 0 0 0 0 0 0 1
0 0 0 0 0 0 1 0 1
0 0 0 0 0 0 0 0 0
### Graph DAG
DFS (discovery and end-processing times):
[ 0] A: dist_time= 1 endp_time=14
[ 1] B: dist_time= 2 endp_time= 9
[ 2] C: dist_time=10 endp_time=13
[ 3] D: dist_time=15 endp_time=16
[ 4] E: dist_time= 3 endp_time= 8
[ 5] F: dist_time=11 endp_time=12
[ 6] G: dist_time= 4 endp_time= 7
[ 7] H: dist_time=17 endp_time=18
[ 8] I: dist_time= 5 endp_time= 6
Topological sort (inverse):
[ 8] [ 6] [ 4] [ 1] [ 5] [ 2] [ 0] [ 3] [ 7]
I   G   E   B   F   C   A   D   H
Topological sort (direct):
[ 7] [ 3] [ 0] [ 2] [ 5] [ 1] [ 4] [ 6] [ 8]
H   D   A   C   F   B   E   G   I
Shortest Paths:
H   D   A   C   F   B   E   G   I
infy infy 0   1   2   1   2   3   2
Longest Paths:
H   D   A   C   F   B   E   G   I
infy infy 0   1   2   1   2   3   4
```

Response

H D A C F B E G I

Exercise 7 (1.5 points)

Given the following directed and weighted graph, apply Dijkstra's algorithm to find all shortest paths connecting node D with all the other nodes. When necessary, consider nodes and edges in alphabetical order.



Report the shortest paths to all vertices. Please indicate the shortest path to all vertices sorted in alphabetic order (i.e., display the shortest path for A B C D, etc.). Report a sequence of integer values separated by one single space. No other symbols must be included in the response. The following is an example of the response: 0 3 2 6 8, etc.

Solution

```
Directed matrix ... stated as such.
Weighted matrix ... stated as such.
Vertex list: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 5] F [ 6] G [ 7] H [ 8] I [ 9] J
Adjacency Matrix:
0 0 0 0 0 3 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 2
0 2 0 0 0 0 0 0 0 0 0
1 0 2 0 3 0 0 1 0 3
0 0 0 0 0 5 5 0 0 0 0
0 0 0 0 0 0 3 0 0 0 0
0 0 0 0 0 0 0 2 4 0 0
0 0 0 0 0 0 0 0 0 0 0
```

```

0 0 0 0 0 0 0 4 0 3
0 0 0 0 0 0 0 0 0 0
### Graph SSSP (Dijkstra and Bellman-Ford)
Dijkstra
Shortest paths from vertex [ 3] D
Node [ 3] D: 0 (pred=-1)
Node [ 0] A: 1 (pred= 3)
Node [ 7] H: 1 (pred= 3)
Node [ 2] C: 2 (pred= 3)
Node [ 4] E: 3 (pred= 3)
Node [ 9] J: 3 (pred= 3)
Node [ 1] B: 4 (pred= 2)
Node [ 5] F: 4 (pred= 0)
Node [ 6] G: 7 (pred= 5)
Node [ 8] I: 11 (pred= 6)
Bellman Ford
Iteration 0: A:1 B:infy C:2 D:0 E:3 F:8 G:8 H:1 I:12 J:3
Iteration 1: A:1 B:4 C:2 D:0 E:3 F:4 G:7 H:1 I:11 J:3
Iteration 2: A:1 B:4 C:2 D:0 E:3 F:4 G:7 H:1 I:11 J:3
Shortest paths from vertex [ 3] D
Node [ 0] A: 1 (pred= 3)
Node [ 1] B: 4 (pred= 2)
Node [ 2] C: 2 (pred= 3)
Node [ 3] D: 0 (pred=-1)
Node [ 4] E: 3 (pred= 3)
Node [ 5] F: 4 (pred= 0)
Node [ 6] G: 7 (pred= 5)
Node [ 7] H: 1 (pred= 3)
Node [ 8] I: 11 (pred= 6)
Node [ 9] J: 3 (pred= 3)

```

Response

1 4 2 0 3 4 7 1 11 3

Exercise 8 (2.0 points)

Analyze the following program and indicate the exact output it generates. Please report the exact program output with no extra symbols.

```

void str(char *dest, char *str1, char *str2, char *str3) {
    while (*str1) {
        if (isupper(*str1)) {
            *dest = *str1;
            dest++;
        }
        str1++;
    }
    while (*str2) {
        if (isdigit(*str2)) {
            *dest = *str2;
            dest++;
        }
        str2++;
    }
    while (*str3) {
        if (islower(*str3)) {
            *dest = *str3;
            dest++;
        }
        str3++;
    }
    *dest = '\0';
}

int main() {
    char *str1 = "hello123WORLD";
    char *str2 = "aBcD123";
    char *str3 = "testCASE";
    char result[100];
    str(result, str1, str2, str3);
    printf("%s", result);
    return 0;
}

```

Response

WORLD123test

Exercise 9 (2.0 points)

The following code snippet illustrates a library manipulating points in the Cartesian space.

```

client.c
// LINE 1
int main (void) {
    point_t p1, p2;
    float d;
    p1 = new ();
    p2 = new ();
}

```

```

    set (p1, 0.0, 0.0, 0.0);
    set (p2, 10.0, 10.0, 10.0);
    d = dist (p1, p2);
    fprintf (stdout, "D = %f\n", d);
    disp (p1);
    disp (p2);
    return 1;
}
pointPublic.h
#ifndef _POINT_PUBLIC
#define _POINT_PUBLIC
#include <stdio.h>
typedef struct point_s *point_t;
point_t new ();
void set (point_t, float, float, float);
float dist (point_t, point_t);
void disp (point_t);
#endif
pointPrivate.h
#ifndef _POINT_PRIVATE
#define _POINT_PRIVATE
// LINE 2
#include <stdlib.h>
#include <math.h>
#include "pointPublic.h"
#endif
point.c
#include "pointPrivate.h"
point_t new () {
    ... allocate new point ...
}
void set (point_t p, float x, float y, float z) {
    ... set coordinates of a new point ...
}
float dist (point_t p1, point_t p2) {
    ... compute the distance between two points ...
}
void disp (point_t p) {
    ... free point p ...
}

```

Indicate which ones of the following statements are correct. Note that incorrect answers imply a penalty in the final score.

1. In Line 1 file `client.c` must include `pointPrivate.h`.
2. File `pointPublic.h` must include `pointPrivate.h`.
3. File `pointPrivate.h` must include `pointPublic.h`.
4. The structure `struct point_s` must be defined in `pointPublic.h`.
5. The structure `struct point_s` must be defined in `pointPrivate.h`.
6. The structure `struct point_s` must be defined in both `pointPublic.h` and `pointPrivate.h`.
7. If in Line 2 we include the library `stdio.h`, the pre-compiler will include it twice, and we will get multiple definitions and a linker error.

Response

1. FALSE.
2. FALSE.
3. TRUE.
4. FALSE.
5. TRUE.
6. FALSE.
7. FALSE.

Exercise 10 (2.0 points)

Analyze the following program and indicate the exact output it generates. Please report the exact program output with no extra symbols.

```

void A(int, int);
void B(int, int);
void C(int, int);
void D(int, int);
int main() {
    A(3, 3);
    return 0;
}
void A(int counterA, int counterB) {
    if (counterA <= 0) return;
    printf("A");
    B(counterB, counterA - 1);
}
void B(int counterB, int counterC) {
    if (counterB <= 0) return;
    printf("B");
    C(counterC, counterB - 1);
    D(counterC - 1, counterB - 1);
}
void C(int counterC, int counterD) {
    if (counterC <= 0) return;
    printf("C");
    D(counterD, counterC - 1);
}
void D(int counterD, int counterA) {
    if (counterD <= 0) return;
    printf("D");
    A(counterA, counterD - 1);
}

```

Response

ABCDABDA

Exercise 11 (3.0 points)

Write the following C function:

```
void lcp (char **string, int n);
```

To find the Longest Common Prefix among strings. The function receives an array of pointers to strings (`string`); `n` indicates the number of strings. Each string has a maximum length of 20 characters. The function finds the longest common prefix among the strings.

For example, if the input strings are the following {`''flower''`, `''flow''`, `''flight''`}, the function must display (on the standard output) `''fl''`.

Write the entire function using standard C libraries but implement all required personal libraries. Modularize the function adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward code, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution

// Only Function lcp had to be implemented

```

#include <stdio.h>
#include <string.h>
void lcp(char **, int); {
int main() {
    char *string[] = {"flower", "flow", "flight"};
    int n = 3;
    lcp(string, n);
    return 0;
}
void lcp(char **string, int n) {
    char prefix[21];
    if (n == 0) {
        printf("\n");
        return;
    }
    strncpy(prefix, string[0], 20);
    prefix[20] = '\0';
    // Compare the prefix with each string in the array
    for (int i = 1; i < n; i++) {
        int j = 0;
        while (prefix[j] != '\0' && string[i][j] != '\0' && prefix[j] == string[i][j]) {
            j++;
        }
        prefix[j] = '\0'; // Truncate the prefix to the common part
    }
    // Output the result
    printf("%s\n", prefix);
}

```

Exercise 12 (5.0 points)

Write the following C function:

```
listB_t *merge (listA_t *head);
```

That receives a list of lists and returns a simple linked list. The main list (pointed by `head`) contains elements of type `listA_t`. Each `listA_t` element contains a pointer to a secondary list of elements of type `listB_t`. Each `listB_t` element stores a string and a pointer to another `listB_t` element. All secondary lists are sorted alphabetically. The function must **merge** all secondary (**already**) sorted linked lists into one single (unique) sorted linked list and return its pointer (of type `listB_t`).

Define the type `listA_t` and `listB_t` to implement the **linked list** with all necessary structure fields.

Write the entire function using standard C libraries but implement all required personal libraries. Modularize the function adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward code, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution

```
// Only function merge should be implemented
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct listA_s listA_t;
typedef struct listB_s listB_t;
struct listA_s {
    listA_t *next;
    listB_t *right;
};
struct listB_s {
    char *str;
    listB_t *next;
};

listB_t *merge (listA_t *);
void pl (listB_t *);

int main() {
    listA_t *a = malloc (4 * sizeof(listA_t));
    listB_t *b = malloc (10 * sizeof(listA_t));
    listB_t *ml;
    setbuf(stdout, 0);
    a[0].next = &a[1];
    a[1].next = &a[2];
    a[2].next = &a[3];
    a[3].next = NULL;
    a[0].right = &b[0];
    a[1].right = &b[2];
    a[2].right = &b[5];
    a[3].right = &b[9];
    b[0].next = &b[1];
    b[1].next = NULL;
    b[0].str = strdup("l1e1");
    b[1].str = strdup("l1e2");
    b[2].next = &b[3];
    b[3].next = &b[4];
    b[4].next = NULL;
    b[2].str = strdup("l2e1");
    b[3].str = strdup("l2e2");
    b[4].str = strdup("l2e3");
    b[5].next = &b[6];
    b[6].next = &b[7];
    b[7].next = &b[8];
    b[8].next = NULL;
    b[5].str = strdup("l3e1");
    b[6].str = strdup("l3e2");
    b[7].str = strdup("l3e3");
    b[8].str = strdup("l3e4");
    b[9].next = NULL;
    b[9].str = strdup("l4e1");
    // Debug
    printf("Single Lists:\n");
    pl (a[0].right);
    pl (a[1].right);
    pl (a[2].right);
    pl (a[3].right);
    ml = merge(&a[0]);
    // Print the merged list
    printf("Merged List:\n");
    pl (ml);
    // Free allocated memory (not shown for simplicity)
```

```

    return 0;
}

listB_t *merge(listA_t *head) {
    listA_t *tmpA, *minA;
    listB_t *tmpB, *finalB;
    int count;
    count = -1;
    finalB = NULL;
    while (count != 0) {
        count = 0;
        minA = NULL;
        tmpA = head;
        while (tmpA != NULL) {
            if (tmpA->right != NULL) {
                printf("(%s)", tmpA->right->str);
                count++;
            }
            if (minA == NULL) {
                minA = tmpA;
            } else {
                if (strcmp(tmpA->right->str, minA->right->str) <= 0) {
                    minA = tmpA;
                }
            }
            tmpA = tmpA->next;
        }
        if (minA != NULL) {
            printf("[%s]{%d}\n", minA->right->str, count);
            tmpB = minA->right;
            minA->right = minA->right->next;
            tmpB->next = finalB;
            finalB = tmpB;
        }
    }
    return finalB;
}

void pl (listB_t *head) {
    listB_t *tmp;
    tmp = head;
    while (tmp != NULL) {
        fprintf (stdout, "%s ", tmp->str);
        tmp = tmp->next;
    }
    fprintf (stdout, "\n");
    return;
}

```

Exercise 13 (9.0 points)

Write a C function that given a set of tasks, each one with a time duration and a priority, selects the subset of those tasks whose total duration time is less or equal to a given threshold while maximizing the sum of the priorities. The prototype of the function is the following:

```
void opt_task (int n, int *time, int *priority, int th, int *best, int *l, int *max);
```

In the prototype, the following are input parameters: Variable `n` is the number of tasks, `time` the array expresses their duration (in hours), and `priority` the array indicates their priorities (or their rewards, once solved). In the prototype, the output parameters are the following: The array `best` specifies the selected tasks (their indices in the `time` and `priority` arrays), the number of tasks assigned and stored into `best`, `l`, and the variable `max` storing the sum of the tasks' priorities (i.e., the total reward obtained from their selection).

For example, let us suppose that $n = 4$, $time = \{2, 3, 4, 5\}$, $priority = \{3, 4, 5, 8\}$, and $th = 7$. The function should select tasks 0 and 3 whose times are 2 and 5, have a sum $2 + 5 = 7 \leq th$, and a total priority (reward) of $3 + 8 = 11$. Thus, it should return $best = \{0, 3\}$, $l = 2$, and $max = 11$.

Function `opt_task` can be considered a wrapper defining other objects (such as the current solution array and size) and calling the actual recursive function (e.g., `opt_task_r`).

Write the entire function using standard C libraries but implement all required personal libraries. Modularize the function adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward code, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution

```

#include <stdio.h>
#define MAX_TASKS 100
void combination(
    int, int *, int *, int, int *, int *, int *, int, int, int
);
int main() {
    int time[] = {2, 3, 4, 5};          // Task completion time

```

```

int priority[] = {3, 4, 5, 8}; // Task priority priority
int n = sizeof(time) / sizeof(time[0]);
int th = 7; // Maximum total completion time
int best_subset[MAX_TASKS]; // Array to store the indices of the best subset
int current_subset[MAX_TASKS]; // Temporary array for recursion
int max = 0; // Variable to store the maximum priority
// Initialize the best subset array
for (int i = 0; i < MAX_TASKS; i++) {
    best_subset[i] = -1;
}
// Find the optimal combination of tasks
combination(n, time, priority, th, best_subset, &max, current_subset, 0, 0, 0, 0);
// Print the result
printf("Best tasks (indices): ");
for (int i = 0; best_subset[i] != -1; i++) {
    printf("%d ", best_subset[i]);
}
printf("\n");
printf("Maximum priority      : %d\n", max);
return 0;
}

void combination(
    int n, int* time, int* priority, int th, int* best_subset, int* max,
    int* current_subset, int current_n, int current_priority, int current_time, int index
) {
    if (current_time > th) {
        // If the current subset exceeds the th, return
        return;
    }
    if (index == n) {
        // If all tasks have been considered, check if the current subset is better
        if (current_priority > *max) {
            *max = current_priority;
            // Copy the current subset to the best subset
            for (int i = 0; i < current_n; i++) {
                best_subset[i] = current_subset[i];
            }
            best_subset[current_n] = -1; // Mark the end of the subset
        }
        return;
    }
    // Case 1: Exclude the current task
    combination(n, time, priority, th, best_subset, max, current_subset, current_n,
        current_priority, current_time, index + 1);
    // Case 2: Include the current task
    current_subset[current_n] = index;
    combination(n, time, priority, th, best_subset, max, current_subset, current_n + 1,
        current_priority + priority[index], current_time + time[index], index + 1);
    return;
}

```